

TP2 – Expressions

Ces TP sont automatiquement évalués à partir d'une archive que vous devez déposer à la date et dans le dépôt prévus se trouvant sur la page Moodle de l'UE. Pour construire cette archive, vous devez taper la commande:

```
$ make tp2
```

Cette commande produit le fichier tp2-DATE.tgz à déposer. Comme les TP se tiennent semaine après semaine, la date de dépôt est positionnée un jour avant la prochaine séance de TP. Pour chaque groupe de TP, un rappel des dates figure juste avant le dépôt. La date de dépôt n'est pas stricte mais vous ne serez évalué et vous ne pourrez avoir un retour sur votre travail que si vous déposez dans les temps.

Attention : *l'usage de l'IA pour faire le TP est formellement interdit !*

Ce TP a pour objectif de mettre en place la procédure pour étendre notre compilateur Logo et d'ajouter le traitement des *variables* et des *expressions arithmétiques*.

Structure du compilateur

Comme exposé durant le cours, les compilateurs sont divisés entre le front-end (qui analyse le programme source) et le back-end (qui génère le code machine). Le front-end peut à son tour être divisé en 3 parties :

1. l'*analyse lexicale* générée par `ocamllex` à partir du fichier `lexer.mll`,
2. l'*analyse syntaxique* générée par `ocamlyacc` à partir du fichier `parser.mly`
3. l'*analyse sémantique* écrite en OCAML et insérée dans les actions de `parser.mly`.

Remarquez que vous n'avez pas vous même à faire appel à ces différents outils mais seulement à utiliser le **Makefile** :

```
$ make
```

L'analyse syntaxique

Le programme principal est l'*analyseur syntaxique* qui fait appel à l'*analyseur lexical* pour obtenir la séquence de *mots*. Le langage de programmation, Logo en l'occurrence, est décrit par une *grammaire*. L'exemple ci-dessous, extrait de `parser.mly`, montre les règles pour les commandes `cmd` et pour les expressions `expr` :

```
cmd:
    FORWARD expr
        { SYSCALL (Logo.cFORWARD, [$2]) }
|    RIGHT expr
        { SYSCALL (Logo.cRIGHT, [$2]) }
;

expr:
    INT
        { CST $1 }
;
```

Les *règles* sont décrites, d'abord, par le nom du *non-terminal*, suivi de “:”, suivi de plusieurs *productions* séparées par “|” et sont terminées par “;”. Chaque *production* est une séquence de *symboles* (*terminal* en majuscule, *non-terminal* en minuscule) et sont complétées par des *actions* entre “{ ... }” en OCAML.

Les *actions* sont déclenchées quand une *production* est réduite durant l'analyse **LALR(1)** générée par `ocamlyacc`. A l'exception des *actions*, la syntaxe de la grammaire est très proche de ce qui a été présenté en cours et peut donc être écrite et comprise facilement.

Les *terminaux* (appelés *tokens* en anglais) sont déclarés en tête du fichier `parser.mly` mais sont reconnus par `lexer.mll` à partir du fichier source. Dans `parser.mly`, la déclaration des non-terminaux est la suivante :

```
%token FORWARD
%token RIGHT
%token<int> INT
```

Les identificateurs de terminaux doivent être en majuscule pour suivre les règles de nommage classique d'OCAML. Ils sont transformés en type somme par `ocamlyacc` :

```
type token =
|    FORWARD
|    RIGHT
|    INT of int
```

A faire : examinez le fichier `parser.mly` pour localiser les éléments présentés dans cette section.

L'analyseur lexical

La tâche de l'analyseur lexical est de lire le texte source pour reconnaître les terminaux et de les renvoyer, avec éventuellement une valeur associée, à l'analyseur syntaxique. Il faut remarquer qu'il n'y a pas de lien direct entre le nom du terminal (dans `parser.mly`) et le mot correspondant (dans `lexer.mll`).

Dans l'exemple ci-dessous, le résultat de lecture du mot-clé `"forward"` correspond au terminal `FORWARD` défini dans `parser.mly`. De même, on voit bien que le caractère `"("` correspond au terminal `LEFT_PAR`. Par conséquent, `ocaml yacc` ne connaît pas le vrai texte correspondant au terminaux mais seulement l'identificateur associé.

```
rule token = parse
|      ...
|      '('          { LEFT_PAR }
|      "forward"    { FORWARD }
|      "right"      { RIGHT }
|      ...
```

Dans `lexer.mll`, chaque *règle* démarre par un `"|"`, est suivie par une *expression régulière* (ER) et terminée par une *action* en OCAML entre `"{ ... }"`. Le rôle de cette *action* est de renvoyer le *terminal* correspondant au *mot* reconnu mais cela peut être n'importe quelle code OCAML renvoyant un *terminal*.

Dans l'exemple ci-dessous, le terminal `INT` est intéressant car il ne permet pas seulement de reconnaître un *mot* mais une famille de *mots* représentant une valeur entière (en décimal). Du point de vue de la *grammaire*, il est suffisant de savoir qu'il s'agit d'un nombre entier. Mais, à un moment donné (typiquement dans le *back-end*), le compilateur va avoir besoin de la valeur de l'entier. On appelle une telle valeur une *valeur sémantique* et elle doit accompagner l'identificateur du *terminal*. Le *type* de cette valeur est donc déclaré dans `parser.mly` et elle doit être fournie lors du retour du *terminal* dans `lexer.mll` :

```
|      ['0'-'9']+  as n      { INT (int_of_string n) }
```

On reconnaît donc un mot composé de un ou plusieurs chiffres décimaux, caractères ASCII entre `'0'` et `'9'` inclus. On associe cette chaîne de caractère avec l'identificateur `n` puis on renvoie le terminal `INT` accompagné de la *valeur sémantique* obtenue en convertissant `n` en entier.

Les ER pouvant être longues à écrire et peu lisibles, `ocamllex` permet de nommer des ER et les réutiliser dans les règles. L'exemple ci-dessus pourrait devenir :

```
let digit = ['0'-'9']
let sign  = ['+' '-']
let dec = sign? digit+
...
|      dec as n      { INT (int_of_string n) }
```

A faire : examinez le fichier `lexer.mll` pour repérer les éléments décrits dans cette section.

Syntaxe des ER en ocamllex

c est un caractère, E une ER.

- ‘ c ’ reconnaît le caractère c .
 - “ $c_1c_2 \dots$ ” reconnaît la suite de caractères $c_1, c_2, \dots$
 - [$c_1 c_2 \dots$] reconnaît un des caractères $c_1, c_2, \dots$. Entre les crochets, ajouter un tiret c_1-c_2 entre deux caractères (par exemple : [‘0’ – ‘9’]) représente tous les caractères de c_1 à c_2 dans l’ordre ASCII.
 - E^* reconnaît une suite de 0 ou plusieurs E .
 - E^+ reconnaît une suite de 1 ou plusieurs E .
 - $E?$ reconnaît 0 ou 1 E .
 - $E_1 E_2 \dots$ reconnaît la séquence $E_1, E_2, \dots$
 - $E_1 | E_2$ reconnaît soit E_1 , soit E_2 .
 - (E) pour gérer les priorités (entre séquence et |).
 - (souligné) reconnaît n’importe quel caractère.
-

Comment étendre notre compilateur ?

En résumé,

1. L’*analyse lexicale* lit les caractères du fichier source, par exemple “123”.
2. Ce texte est accepté par l’Automate Fini Déterministe (AFD) correspondant à `dec`.
3. L’action associée est alors appelée avec $n = \text{“123”}$.
4. Elle crée un terminal `INT` associé à la valeur sémantique 123 et le renvoie à l’*analyse syntaxique*.
5. L’*analyse syntaxique* utilise ce *terminal* pour avancer son travail, c’est-à-dire l’empile.
6. Quand suffisamment de *terminaux* ont été accumulés pour reconnaître une *production*, l’action correspondant à la *réduction* de la *production* est appelée. Par exemple, `INT` fait partie de la règle `FORWARD INT` qui lance de la construction de la représentation intermédiaire de ce code.

En conséquence, pour étendre notre langage, on doit :

1. Déclarer les *terminaux* nécessaires dans `parser.mly`.
2. Ajouter les *règles de grammaire* dans `parser.mly`.
3. Vérifier si cela compile et que la *grammaire* est LALR(1) (en appelant `make`).
4. Ajouter la reconnaissance des nouveaux *terminaux* dans `lexer.mll`.
5. Vérifier que tout compile correctement.
6. Tester que les nouvelles *règles* fonctionnent bien avec un programme les utilisant.

Par exemple, pour vérifier que le compilateur Logo fonctionne sur le programme `square.logo`, on tapera :

```
$ ./logocc progs/square.logo
Assembly saved to progs/square.s
```

L'assembleur sera généré dans **progs/square.s**.

Cependant, si mon extension¹ ne fonctionne pas correctement, le compilateur pourra émettre un message du type :

```
$ ./logocc progs/cross.logo
ERROR:3:1: illegal char 'x'
```

A faire : tester les commandes ci-dessus.

Premiers pas

Pour l'instant, notre compilateur supporte les programmes comportant des commandes **forward** et **right** prenant comme paramètre un simple entier. Nous allons maintenant ajouter le support pour la commande **backward** qui prend une expression en paramètre.

1. Il faut d'abord ajouter la déclaration du terminal **BACKWARD** dans l'en-tête de **parser.mly** (avec les autres définitions de terminaux) :

```
%token BACKWARD
```

2. Où est-ce qu'une telle commande peut être utilisée dans notre langage ? ~> Au même niveau que les commandes **forward** et **right**, c'est-à-dire qu'il s'agit d'une production du non-terminal **cmd**. On ajoute donc la production :

```
| BACKWARD expr
  { NOP }
```

L'action est positionnée à **NOP** car, dans ce TP, on ne s'intéresse pas à générer du code : on veut seulement reconnaître la commande et on génère l'instruction vide avant de s'intéresser à la génération de code dans un prochain TP. C'est ce qu'on Cela nous permet de rapidement tester la syntaxe. On peut alors construire le compilateur (et corriger les erreurs s'il y en a) :

```
$ make
```

3. Le travail est terminé au niveau de l'analyse syntaxique. Nous devons maintenant réaliser l'analyse lexicale. Ouvrez **lexer.mll** et ajoutez la reconnaissance du mot-clé **backward** :

```
| "backward" { BACKWARD }
```

Vérifiez que tout compile.

4. Testez votre extension sur **progs/forwardback.logo**.

¹ c'est-à-dire, les ajouts que je fais dans le lexeur ou la grammaire.

IMPORTANT : le flot de caractères en entrée est traité pour mettre toutes les lettres en minuscules afin de rendre notre implantation insensible à la casse. Par conséquent, il faut donner le nom des commandes en minuscule.

A faire : appliquez cette méthode pour développer la commande `left` qui prend elle-aussi une expression en paramètre et `home` qui ne prend pas de paramètre. Testez votre implantation avec `progs/cross.logo`.

Résumé : pour étendre le compilateur Logo, il faut (a) ajouter les nouveaux *terminaux* dans `parser.mly` et leur reconnaissance dans `lexer.mll`, (b) écrire les nouvelles *règles* de *grammaire* dans `parser.mly` (en ajoutant des *actions nulles*) et (c) pour tester le tout, utiliser des fichiers de test contenant les nouvelles formes de *grammaire* de notre *langage*.

NOTE : les actions nulles sont `NOP` pour les `cmd` et `NONE` pour les `expr`.

Débogage : afin de déboguer votre *grammaire* et observer, par exemple, l'ordre des *réductions*, vous pouvez utiliser dans les *actions* les commandes d'affichage d'OCAML : `print_string`, `print_int`, `printf`, etc. Cependant, gardez à l'esprit le fonctionnement de l'algorithme LALR(1) pour éviter les méprises.

Variables

Comme de nombreux langages, Logo supporte le concept de variable. Cependant, la syntaxe peut sembler étrange.

L'exemple ci-dessous montre comment on définit une variable nommée `x` avec l'instruction `MAKE` (en préfixant l'identificateur avec `""`) et comment on utilise la valeur d'une variable avec le préfixe `:"`.

```
MAKE "x 111
PRINT :x
```

NOTE : l'instruction `PRINT` a déjà été implantée par nos soins dans `parser.mly`.

Il existe bien sûr plusieurs manières pour implanter une telle syntaxe mais on va choisir que les identificateurs arrivent sous 2 formes :

- le terminal **NAME** représente les variables à leur déclaration (préfixé par `""`),
- le terminal **REF** représente les variables lorsqu'elles sont utilisées (préfixé par `:"`).

Dans tous les cas, elles prendront comme valeur sémantique, l'identifiant de la variable non préfixé. Un identificateur de variable est défini de manière assez classique : il doit commencer par une lettre (minuscule ou majuscule) ou un souligné suivi de 0 ou plusieurs lettres, soulignés ou chiffres.

NB : dans les expressions régulières d'`ocamllex` (le langage pour écrire `lexer.mll`), `"r as x"` permet d'accéder par la définition `x` au mot reconnu par `r`. On peut le faire pour toute

l'expression ou pour certaines parties en l'isolant entre parenthèses : " $r_1 (r_2 \text{ as } x)$ " reconnaît les mots de l'ER $r_1.r_2$ mais x ne vaut que le sous-mot reconnu par r_2 .

A faire (à chaque étape, on fera au moins une compilation)

1. Définissez dans `parser.mly` les terminaux **NAME**, **REF** et **MAKE**.
2. Exprimez avec des expressions régulières ces terminaux dans `lexer.mll`.
3. Écrivez les productions pour l'affectation de variable dans **cmd** dans `parser.mly`.
4. Écrivez les règles permettant d'utiliser la valeur d'une variable (sous-question : comment vous les connectez au reste de la grammaire ?)
5. Testez votre code avec `progs/make.logo`.
6. Il est aussi intéressant de tester les cas d'erreur, par exemple, avec `progs/make-err.logo`.

Addition

Nous allons maintenant introduire l'opération d'addition qui est notée de manière infixé avec comme opérateur le classique "+". Les arguments d'une addition peuvent être :

- un entier,
- une référence à une variable,
- une sous-expression entre "...",
- une autre addition.

Dans ce dernier cas, on notera que l'addition est, comme dans la plupart des langages de programmation, associative à gauche. Ainsi, " $1 + 2 + 3$ " doit être interprété " $(1 + 2) + 3$ ".

A faire : on va appliquer une méthode progressive pour ajouter des extensions et les tester.

1. Ajoutez les *terminaux* et implémentez les *règles de grammaire* de l'addition sur les entiers et les variables dans `parser.mly` et `lexer.mll`.
2. Ouvrez et observez `progs/add.logo` puis testez notre compilateur sur ce fichier (seule une partie du fichier doit être analysée sans erreurs).
3. Ajoutez la syntaxe avec parenthèse "(...)" (modifiez `parser.mly` et `lexer.mll`).
4. Implémentez la forme avec plusieurs symboles "+", " $1 + 2 + 3$ ", qui se résout par de l'*associativité à gauche*.

Pour vérifier l'*associativité à gauche*, on peut exploiter l'*ordre de réduction* LR(k) et l'affichage dans OCAML. En LR(k), les *réductions* sont effectuées du bas vers le haut. Les arguments sont donc réduits avant les opérations qui les utilisent.

La solution proposée est de mettre un affichage dans chaque *action* pour voir l'élément entier, le nom de variable ou l'addition "+" reconnue. Les valeurs sémantiques sont accessibles avec l'identificateur spécial \$1 :

```
| INT { printf "%d " $1; CST $1 }
```

Ainsi, l'expression " $1 + 2 + 3$ " devrait afficher " $1\ 2 + 3 +$ " (en fait, on obtient la même expression en notation polonaise inversée).

NB : `ocamlyacc` (le langage pour écrire `parser.mly`) supporte aussi des commandes spéciales `%left`, `%right` et `%nonassoc` que nous n'utiliserons pas durant ce TP.

Implantation des autres opérations

Il reste maintenant à faire de même avec les autres opérations :

- soustraction notée “-”,
- multiplication notée “*”,
- division notée “/”,
- modulo noté “%”,
- puissance notée “^”.

On notera les propriétés suivantes :

- “+”, “-”, “*”, “/” et “%” sont *associatifs à gauche*.
- “^” est *associatif à droite*,
- “+” et “-” ont même *priorité* et sont les opérateurs les moins prioritaires,
- “*”, “/” et “%” ont même *priorité* et sont prioritaires sur “+” et “-” mais moins prioritaire que “^”.
- “^” est plus prioritaire que tous les autres opérateurs.

Ainsi, “1 * 2 + 3”, complètement parenthésé, équivaut à “(1 * 2) + 3”. De même, “1 + 2 ^ 3” équivaut “1 + (2 ^ 3)”.

Pour gérer les priorités, on pourra exploiter l'*ordre de réduction des grammaires LR(k)*, du bas vers le haut. Ainsi, une opération placée plus bas (plus éloignée de l'axiome) dans la grammaire sera réduite avant une opération placée plus haut. Vous pouvez vous inspirer de la grammaire LR(k) du mini-C du cours. Pour le retour de ces actions (qui doivent être des expressions), on retournera **NONE** pour l'instant.

A faire : testez avec le fichier `progs/expr.logo`.

1. Ajoutez la soustraction “-”.
2. Ajoutez la multiplication “*”.

On pourra vérifier associativité et priorité de la même manière que pour l'addition dans la section précédente. Par exemple, “1 + 2 * 3” doit afficher “1 2 3 * +”.

3. Ajoutez la division “/” et le modulo “%”.
4. Ajoutez l'opérateur de puissance “^”.

Attention : pour l'instant, il n'est pas possible de faire vraiment la compilation. On se concentre sur l'analyse lexicale et syntaxique sans faire la génération de code. Par conséquent, il est normal que le lancement de `logocc` se termine par un `Failure("attempt to compile NONE expression!")`.